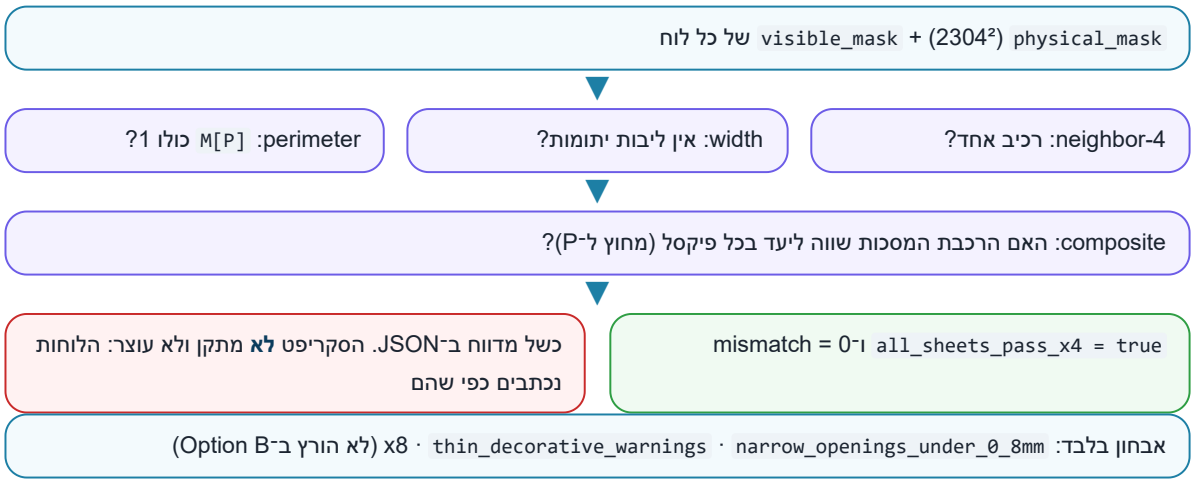
10. אימות פיזי

ה־validator **לא נכתב מחדש**. בכל לוח שהמתכנן מייצר, הסקריפט קורא לפונקציות הקיימות מ־`connectivity_repair.py` בלי שינוי (`scripts/run_turtle_reveal_planner_poc.py:94`). המתכנן משתמש באותו גרעין ובאותה הגדרת עיגון, כך שה־validator הוא בדיקה בלתי תלויה של אותה הגדרה ולא של הגדרה אחרת.

10.1 תרשים 7: זרימת האימות והטיפול בכשל



10.2 בדיקה 1: קשירות 4 (audit_edge_connectivity)

`src/layerforge_l1/connectivity_repair.py:82`

```python
# src/layerforge_l1/connectivity_repair.py:85-108
    binary = mask.astype(np.uint8)
    count4, labels4, stats4, _ = cv2.connectedComponentsWithStats(
        binary, connectivity=4
    )
    border4 = set(
        int(value)
        for value in np.unique(
            np.concatenate(
                (labels4[0], labels4[-1], labels4[:, 0], labels4[:, -1])
            )
        )
        if value
    )
    orphans4 = [component for component in range(1, count4) if component not in border4]
    count8, _, _, _ = cv2.connectedComponentsWithStats(binary, connectivity=8)
    report = {
        "eight_neighbor_components": count8 - 1,
        "four_neighbor_components": count4 - 1,
        "four_neighbor_perimeter_components": len(border4),
        "four_neighbor_orphan_components": len(orphans4),
        "four_neighbor_orphan_pixels": int(
            sum(stats4[component, cv2.CC_STAT_AREA] for component in orphans4)
        ),
        "four_neighbor_status": "PASS" if count4 - 1 == 1 else "FAIL",
```

IMPLEMENTED MATHEMATICS

$$\text{PASS} \iff \#\{\text{4-connected components of } M\} = 1$$

מה זה מודד: אם כל החומר בלוח הוא חתיכה אחת. שני פיקסלים שנוגעים רק בפינה (אלכסון) לא נחשבים מחוברים. זו הגדרה שמרנית, כי בהדפסה מגע בפינה הוא נקודה בלי שטח. הדוח מוסיף ספירה ב־8 שכנים ו"יתומים", כלומר רכיבים שלא נוגעים בשולי התמונה, למטרות אבחון בלבד.

מה זה לא מוכיח: חוזק. חתיכה אחת יכולה להיות מחוברת בצוואר של פיקסל אחד. את זה תופסת בדיקה 2.

10.3 בדיקה 2: רוחב מבני (audit_connection_width)

`src/layerforge_l1/connectivity_repair.py:227` ו־`src/layerforge_l1/connectivity_repair.py:152`

src/layerforge_l1/connectivity_repair.py:152-165

```
152  def _core_connectivity(
153      material: np.ndarray, perimeter: np.ndarray, kernel_size: int
154  ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, list[int]]:
155      kernel = cv2.getStructuringElement(
156          cv2.MORPH_ELLIPSE, (kernel_size, kernel_size)
157      )
158      core = cv2.erode(material, kernel, iterations=1).astype(bool)
159      count, labels, stats, _ = cv2.connectedComponentsWithStats(
160          core.astype(np.uint8), connectivity=4
161      )
162      anchored = set(int(value) for value in np.unique(labels[core & perimeter]) if value)
163      orphan_ids = [component for component in range(1, count) if component not in anchored]
164      connected = np.isin(labels, list(anchored))
165      return core, connected, labels, stats, orphan_ids
```

src/layerforge_l1/connectivity_repair.py:247-257

```
247      kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel_size, kernel_size))
248      material = physical_mask.astype(np.uint8)
249      _, perimeter = _outer_and_perimeter(poc)
250      core, connected_core, core_labels, stats, orphan_core = _core_connectivity(
251          material, perimeter, kernel_size
252      )
253      anchored = set(int(value) for value in np.unique(core_labels[connected_core]) if value)
254      restored_support = (
255          cv2.dilate(connected_core.astype(np.uint8), kernel, iterations=1).astype(bool)
256          & physical_mask
257      )
```

IMPLEMENTED MATHEMATICS

$$\text{core} = M \ominus K_{13}, \qquad \text{Orph} = \text{core} \setminus \text{Anch}_{P_0}(\text{core}), \qquad \text{PASS} \iff \text{Orph} = \emptyset$$

כאן היא הטבעת בלבד (P_0) `_outer_and_perimeter(poc)`. פסי ה־magenta לא כלולים בה, אבל הם מחוברים לטבעת ונשחקים לליבה מחוברת, ולכן התוצאה זהה. `src/layerforge_l1/connectivity_repair.py:301`. הסטטוס בכשל הוא `CONFLICT_REQUIRES_USER_DECISION`.

DERIVED **מה PASS מוכיח:** כל חלק בלוח שמספיק עבה כדי להכיל אליפסה של 1.27 מ"מ מחובר למסגרת דרך מסדרון ברוחב האליפסה (מסלול ליבה 4-קשיר, פרק 7.2). יחד עם בדיקה 1, החלקים הדקים (שאין בהם ליבה) מחוברים פיזית, אבל לא בהכרח במסדרון רחב.

מה PASS לא מוכיח: (א) שחלקים דקים, בלי ליבה, קצרים. ה-validator רק מסווג אותם כ-`thin_decorative` (אזהרה).

ה**מתכנן** הוא שמגביל אותם ל-0.5 מ"מ גאודזית. (ב) חוזק מכני, עיוות או התאמה לחומר ההדפסה. (ג) פתחים צרים בחומר (ראה בדיקה 5).

סיווג רכיבים גלויים (אבחון)

```
src/layerforge_l1/connectivity_repair.py:270-281
270      orphan_core_mask = np.isin(core_labels, orphan_core)
271      thin_decorative: list[int] = []
272      structural_bottleneck_visible: list[int] = []
273      structurally_supported_visible: list[int] = []
274      for component in range(1, visible_count):
275          component_mask = visible_labels == component
276          if not np.any(core & component_mask):
277              thin_decorative.append(component)
278          elif np.any(orphan_core_mask & component_mask):
279              structural_bottleneck_visible.append(component)
280          else:
281              structurally_supported_visible.append(component)
```

כל רכיב 4-קשיר של V מסווג לאחת משלוש קטגוריות: **thin_decorative** (אין בו אף פיקסל ליבה), **structural_bottleneck_visible** (חלק מליבה יתומה) או **structurally_supported**. ב-Option B: 0 אזהרות thin_decorative בכל 10 הלוחות (OBSERVED).

מדידת "צוואר הבקבוק" ליתומים (אבחון)

`src/layerforge_l1/connectivity_repair.py:168`: לכל ליבה יתומה הפונקציה בודקת את כל הגרעינים האי-זוגיים מ-1 עד 13, ומדווחת על הגדול ביותר שבו האזור עדיין מחובר. התוצאה היא "חוסר רוחב" בדיד: $(13 - k^*) \cdot s$ מ"מ. ב-Option B לא היו יתומים, ולכן החלק הזה לא הופעל.

10.4 בדיקה 3: מסגרת משותפת

```
scripts/run_turtle_reveal_planner_poc.py:128-128
128                  "shared_perimeter_complete": bool(physical[planner.perimeter].all()),
```

כל הפיקסלים של `planner.perimeter`, כלומר הטבעת ופסי המסגרת, חייבים להיות חומר. זה מבטיח שכל הלוחות זהים בהיקף החיצוני.

10.5 בדיקה 4: שחזור מדויק

```
scripts/run_turtle_reveal_planner_poc.py:134-136
134      target = embed(labels.astype(np.int16), planner.grid, -1)
135      inside = ~planner.perimeter  # includes any frame padding
136      row["reconstruction_mismatched_pixels_x4"] = int((composite[inside] != target[inside]).sum())
```

IMPLEMENTED MATHEMATICS ההרכבה נבנית בשורה 93: `composite[(composite < 0) & physical] = color`, לפי סדר מלפנים לאחור:

$$\mathrm{comp}(p) = c_{k^*(p)}, \quad k^*(p) = \min\{k:\ p \in M_k\}, \qquad \text{mismatch} = \left|\{p \notin P:\ \mathrm{comp}(p) \neq \mathrm{target}(p)\}\right|$$

ב־Option B: `reconstruction_mismatched_pixels_x4 = 0`. זה אישור אמפירי למשפט מסעיף 8.1.

10.6 בדיקה 5: פתחים צרים (אבחון בלבד)

```python
# scripts/run_turtle_reveal_planner_poc.py:43-49
def _narrow_openings(physical: np.ndarray, mm_per_pixel: float) -> int:
    holes = (~physical).astype(np.uint8)
    size = int(np.ceil(OPENING_MM / mm_per_pixel))
    size += 1 - size % 2
    opened = cv2.morphologyEx(holes, cv2.MORPH_OPEN, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (size, size)))
    count, labels = cv2.connectedComponents(holes, connectivity=4)
    return int(sum(1 for c in range(1, count) if not opened[labels == c].any()))
```

IMPLEMENTED MATHEMATICS

$$H = \overline{M}, \quad k = \lceil 0.8/s \rceil = 9\ (\text{odd}), \quad \text{count} = \left|\{Q\ 4\text{-comp of } H:\ Q \cap (H \circ E_9) = \emptyset\}\right|$$

סופר חורים שאי אפשר להכניס לתוכם דיסקה של 0.88 מ"מ, כלומר חור צר שעלול להיסגר בהדפסה. ב־Option B היה חור אחד כזה בכל אחד מ־4 הלוחות הקדמיים. NOT INVESTIGATED זה רק דיווח, לא חלק מ־PASS/FAIL, ולא נבדק אילו חורים אלה.

10.7 אימות ברזולוציה כפולה (x8): לא הורץ

הסקריפט יכול להריץ את אותם validators אחרי הגדלה של פי 2 בשיטת nearest (`src/layerforge_l1/hidden_width_repair.py:65`). ב־Option B הוא הורץ עם `--skip-x8` כדי לחסוך זמן וזיכרון, ולכן בדוח מופיע `NOT_RUN`. GAP

10.8 דיוק, דיסקרטיזציה וטולרנסים

- כל המדידות בדידות, ברזולוציה של $s = 0.0977$ מ"מ. רוחב "1.2 מ"מ" נאכף בפועל כ־13 פיקסלים, כלומר 1.27 מ"מ.
- הגרעין האליפטי מקרב דיסקה. בכיוונים אלכסוניים הוא מעט צר יותר (פרק 6.2).
- הקשר בין רשת היעד לרשת הפיזית הוא 1:1, ולכן אין שגיאת דגימה בין התכנון לאימות.
- המעבר ל־CAD מוסיף עיגול של 0.03 מ"מ (פשטות פוליגונים). לפי בדיקת הרסטור החוזר, ההשפעה עליו היא 0 פיקסלים (פרק 11).